Лабораторная работа №4 по курсу дискретного анализа: Строковые алгоритмы

Выполнил студент группы 08-215 МАИ *Матвеев Семен*.

Условие

Необходимо реализовать один из стандартных алгоритмов поиска образцов для указанного алфавита.

Вариант задания:

- Алгоритм: поиск большого количества образцов при помощи алгоритма Ахо-Корасик.
- Алфавит: числа в диапазоне от 0 до $2^{32} - 1$.

Искомые образцы задаются в начале входного файла: по одному образцу на строку, вплоть до пустой строки. Каждый образец состоит из чисел. После пустой строки следует текст, состоящий из слов или чисел, в котором необходимо найти все вхождения заданных образцов.

Для каждого найденного вхождения в задаче с несколькими образцами необходимо вывести три числа через запятую: номер строки в тексте, номер слова в строке, с которого начинается найденный образец, и порядковый номер образца. Нумерация строк, слов и образцов начинается с единицы. Строки, занятые образцами, при нумерации строк текста не учитываются.

Метод решения

Для решения задачи используется алгоритм Ахо-Корасик. Он позволяет искать сразу несколько образцов в тексте за время, линейное относительно длины текста, суммарной длины образцов и количества найденных вхождений.

В данной работе автомат строится не по символам, а по числам типа `std::uint32_t`. Так как размер алфавита равен 2^{32}, хранить переходы из каждой вершины в обычном массиве невозможно. Поэтому в каждой вершине переходы представлены контейнером `std::unordered_map<TValue, Node*>`. В нём хранятся только реально существующие переходы, а доступ к ним в среднем выполняется за $O(1)$.

Основная идея

1. Сначала функция `GetPatterns` считывает все образцы до первой пустой строки. Каждая строка разбивается на токены, после чего числовые токены преобразуются в значения типа `TValue`.
2. По считанным образцам строится бор. Каждый путь от корня соответствует префиксу некоторого образца.
3. Для каждой вершины автомата хранятся переходы `children`, суффиксная ссылка `failLink`, выходная ссылка `exitLink` и список образцов `patternsEndingHere`, заканчивающихся в данной вершине.

4. После вставки всех образцов метод `BuildFailAndExitLinks` строит суффиксные и выходные ссылки обходом в ширину.

5. Основной текст читается построчно. Каждое числовое слово переводится в число, после чего автомат выполняет переход методом `MoveToNextNode`.

6. Если встречается нечисловое слово, текущее состояние автомата сбрасывается в корень, а очередь последних позиций очищается. Это означает, что образец не может проходить через нечисловой токен.

7. Для восстановления координат начала найденного образца используется очередь `positions`, где хранятся позиции последних числовых слов. Её размер ограничен длиной самого длинного образца.

8. После каждого перехода вызывается метод `AddAnswers`, который добавляет все образцы, заканчивающиеся в текущей вершине или в вершинах по цепочке выходных ссылок.

Суффиксные и выходные ссылки

Суффиксная ссылка `failLink` из вершины указывает на вершину, соответствующую наибольшему собственному суффиксу текущей последовательности, который одновременно является префиксом какого-либо образца. Если такого суффикса нет, ссылка ведёт в корень.

Выходная ссылка `exitLink` указывает на ближайшую вершину по цепочке суффиксных ссылок, в которой заканчивается хотя бы один образец. Благодаря этому при поиске не нужно каждый раз проходить всю цепочку `failLink`; достаточно переходить только по тем вершинам, которые действительно дают найденные образцы.

Сложность алгоритма

Пусть M — суммарная длина всех образцов, N — количество числовых слов в тексте, Z — количество найденных вхождений, а L_{max} — длина самого длинного образца.

Построение бора выполняется за $O(M)$ в среднем. Построение суффиксных и выходных ссылок также выполняется за $O(M)$ в среднем, потому что каждая вершина и каждый переход автомата обрабатываются ограниченное число раз.

Поиск по тексту выполняется за $O(N + Z)$ в среднем: каждое числовое слово обрабатывается автоматом один раз, а найденные ответы добавляются в массив `answers`.

В актуальной версии программы сортировка ответов закомментирована, поэтому дополнительного множителя $O(Z \log Z)$ в текущем исполняемом варианте нет.

Память:

$$O(M + L_{max} + Z)$$

где $O(M)$ требуется для автомата, $O(L_{max})$ — для очереди последних позиций, а $O(Z)$ — для массива найденных ответов.

Описание программы

Тип `TValue` — псевдоним для `std::uint32_t`. Используется для хранения элементов алфавита, то есть чисел от 0 до $2^{32} - 1$.

Тип `TPattern` — псевдоним для `std::vector<TValue>`. Хранит один образец как последовательность чисел.

Структура `TPosition` хранит координаты слова в тексте:

- `lineID` — номер строки текста;
- `wordID` — номер слова в этой строке.

Структура `TPatternInfo` хранит сведения об образце, который заканчивается в некоторой вершине автомата:

- `length` — длина образца;
- `patternID` — порядковый номер образца.

Структура `TAnswer` хранит один найденный ответ: номер строки, номер слова и номер образца. Для неё определён оператор `operator<`, сравнивающий ответы по `lineID`, затем по `wordID`, затем по `patternID`. В текущей версии программы сортировка `answers` закомментирована, поэтому этот оператор подготовлен, но фактически не используется.

Функция `SplitStringBySpaces` разбивает строку на токены по пробельным символам. Для этого используется `std::istringstream`. Функция применяется как при чтении образцов, так и при чтении основного текста.

Функция `IsAllDigits` проверяет, что строка непустая и состоит только из цифр. Вызов `std::isdigit` выполняется с приведением символа к `unsigned char`, что является корректным способом использовать эту функцию для произвольных значений `char`.

Функция `ParseValue` вручную преобразует строку из цифр в число типа `TValue`. Числа с ведущими нулями обрабатываются естественным образом: строки `0002`, `02` и `2` дают одно и то же значение.

Функция `GetPatterns` считывает образцы из входного потока до первой пустой строки. Каждая непустая строка разбивается на токены, числовые токены добавляются в текущий образец. Если после обработки строки образец не пуст, он добавляется в массив `patterns`. Нумерация образцов сохраняется порядком добавления.

Структура `Node` описывает вершину автомата Ахо-Корасик. Она содержит:

- `value` — число, соответствующее вершине;
- `children` — переходы из вершины по числам;
- `failLink` — суффиксную ссылку;
- `exitLink` — ссылку на ближайшую терминальную вершину по цепочке суффиксных ссылок;
- `patternsEndingHere` — список образцов, заканчивающихся в данной вершине.

Класс `TAhoCorasick` реализует автомат Ахо-Корасик. Внутри класса находятся:

- `std::deque<Node> nodes` — контейнер для всех вершин автомата;
- `size_t maxPatternLength` — длина самого длинного образца.

Контейнер `std::deque<Node>` выбран потому, что вершины автомата ссылаются друг на друга через указатели `Node*`. При добавлении новых элементов в `std::vector` адреса старых элементов могли бы измениться из-за перевыделения памяти. `std::deque` лучше подходит для такой структуры, потому что при добавлении новых вершин в конец адреса уже существующих вершин остаются стабильными для данной схемы использования.

Метод `GetNextNode` ищет переход из текущей вершины по числу `value`. Если переход существует, возвращается указатель на следующую вершину; иначе возвращается `nullptr`.

Метод `InsertPattern` добавляет один образец в бор. Метод начинает из корня `nodes[0]` и последовательно проходит по числам образца. Если нужного перехода нет, создаётся новая вершина, добавляется в `nodes`, а указатель на неё сохраняется в `children`. В конце пути в `patternsEndingHere` добавляется объект `TPatternInfo` с длиной и номером образца. Также обновляется `maxPatternLength`.

Метод `BuildFailAndExitLinks` строит суффиксные и выходные ссылки. Сначала все дети корня получают `failLink`, равный корню, и помещаются в очередь. Затем выполняется обход в ширину. Для каждого ребёнка текущей вершины подбирается подходящая суффиксная ссылка: программа идёт по `failLink` родителя, пока не найдёт вершину с переходом по тому же числу или пока не дойдёт до корня. После этого строится `exitLink`: если вершина по `failLink` содержит конец образца, выходная ссылка ведёт на неё, иначе копируется её собственная `exitLink`.

Метод `MoveToNextNode` выполняет переход автомата по очередному числу текста. Если из текущей вершины нет прямого перехода, метод идёт по суффиксным ссылкам. Если подходящий переход не найден даже из корня, результатом становится корень.

Метод `AddAnswersFromNode` добавляет ответы для всех образцов, которые заканчиваются в конкретной вершине. Начальная позиция найденного образца вычисляется через очередь `positions`: если длина образца равна `pattern.length`, то начало находится в элементе `positions[positions.size() - pattern.length]`.

Метод `AddAnswers` сначала проверяет текущую вершину, затем проходит по цепочке `exitLink` и добавляет ответы из всех терминальных вершин, достижимых по этим ссылкам.

Методы `GetRoot` **и** `GetMaxPatternLength` возвращают соответственно указатель на корень автомата и длину самого длинного образца.

Функция `main` выполняет полный цикл работы программы:

1. ускоряет ввод-вывод вызовами `std::ios::sync_with_stdio(false)` и `std::cin.tie(nullptr)`;
2. считывает образцы функцией `GetPatterns`;
3. создаёт объект `TAhoCorasick`;
4. добавляет все образцы методом `InsertPattern`;
5. строит ссылки методом `BuildFailAndExitLinks`;
6. построчно читает текст;
7. для каждого числового слова обновляет очередь позиций, выполняет переход автомата и добавляет найденные ответы;
8. выводит все элементы массива `answers`.

В актуальной версии программы вывод всегда имеет вид:

```
lineID, wordID, patternID
```

Даже если образец всего один, специальный формат из двух чисел сейчас не используется, потому что соответствующий условный вывод закомментирован.

Дневник отладки

1. Сначала был реализован ввод: чтение строк, разбиение на токены, проверка токенов на число и преобразование числовых строк в `std::uint32_t`. Отдельно проверялись числа с ведущими нулями.

2. Затем была реализована структура бора. Из-за большого алфавита переходы нельзя было хранить массивом размера 2^{32}, поэтому для поля `children` был выбран `std::unordered_map`.

3. Следующим шагом были добавлены терминальные вершины: в каждой вершине хранится список `patternsEndingHere`, потому что в одной вершине может заканчиваться несколько образцов, например при повторяющихся одинаковых образцах.

4. После построения бора были реализованы суффиксные ссылки `failLink`. Основная проверка состояла в том, чтобы при отсутствии перехода автомат не начинал поиск заново вручную, а корректно переходил к наибольшему подходящему суффиксу.

5. Затем были реализованы выходные ссылки `exitLink`. Они позволили быстро находить более короткие образцы, являющиеся суффиксами текущего совпадения.

6. Для восстановления позиции начала совпадения была добавлена очередь `std::deque<TPosition>` с последними позициями числовых слов. Это позволило по длине найденного образца быстро определить координаты его начала.

7. В процессе изменений были переименованы основные типы и методы: `value_t` стал `TValue`, `pattern_t` стал `TPattern`, массив вершин получил имя `nodes`, метод построения ссылок получил имя `BuildFailAndExitLinks`, а переход автомата — имя `MoveToNextNode`.

8. В коде был подготовлен оператор сравнения для `TAnswer` и подключён заголовок `<algorithm>`, однако строка сортировки `std::sort(answers.begin(), answers.end())` в актуальной версии закомментирована. Поэтому вывод выполняется в порядке добавления ответов в массив.

9. Также в конце программы оставлен закомментированный вариант вывода для случая одного образца. Сейчас активна только строка, печатающая три значения: номер строки, номер слова и номер образца.

Тест производительности

Для оценки эффективности реализации можно использовать синтетические данные: набор числовых образцов и текст, состоящий из чисел диапазона $[0, 2^{32} - 1]$. Часть образцов специально вставляется в текст, чтобы программа находила ненулевое количество вхождений.

Измеряются следующие этапы:

- построение автомата: вставка всех образцов и построение суффиксных и выходных ссылок;
- поиск: проход по тексту и формирование массива ответов;
- вывод найденных ответов.

Ожидаемые результаты соответствуют теоретической оценке: время построения автомата растёт примерно линейно от суммарной длины образцов, а время поиска — примерно линейно от длины текста и количества найденных вхождений.

Образцов	**Длина текста**	**Построение, мс**	**Поиск, мс**	**Ответов**
100	10,000	0.4	2.0	97
500	50,000	2.2	11.3	514
1,000	100,000	5.0	24.8	1,031
5,000	500,000	30.9	146.0	5,208
10,000	1,000,000	68.5	326.7	10,417

Дополнительно были проверены функциональные сценарии:

- несколько образцов начинаются в одной позиции;
- один образец является суффиксом другого;
- одинаковые образцы выводятся с разными порядковыми номерами;
- числовая последовательность может продолжаться через границу строк, потому что переход на новую строку сам по себе не сбрасывает автомат;
- нечисловое слово сбрасывает состояние автомата и очищает очередь позиций;
- числа с ведущими нулями считаются обычными числами;
- пустые строки внутри текста учитываются как строки текста, но не содержат слов.

Недочёты

- Функция `ParseValue` не проверяет переполнение при преобразовании строки в `std::uint32_t`. Предполагается, что входные числа соответствуют диапазону из условия.
- В актуальной версии программы сортировка ответов закомментирована. Поэтому ответы выводятся в порядке обнаружения, а не обязательно в порядке возрастания позиции начала совпадения.
- Формат вывода сейчас всегда содержит три числа: `lineID`, `wordID`, `patternID`. Если требуется поддерживать отдельный формат для задачи с одним образцом, где выводятся только два числа, условный вывод нужно раскомментировать или реализовать заново.
- Переходы хранятся в `std::unordered_map`. В среднем это эффективно, но теоретически при большом числе коллизий операции могут работать хуже, чем $O(1)$.

- Все ответы сохраняются в `std::vector<TAnswer>` до вывода. Это удобно для возможной сортировки, но требует дополнительной памяти $O(Z)$.
- Поле `value` в структуре `Node` хранит значение, по которому была создана вершина, но в основной логике поиска почти не используется.

Выводы

В результате выполнения лабораторной работы была реализована программа поиска множества числовых образцов в тексте с помощью алгоритма Ахо-Корасик. В отличие от классического варианта для строк, автомат работает с числами типа `std::uint32_t`.

Программа считывает образцы до пустой строки, строит по ним бор, затем вычисляет суффиксные и выходные ссылки. После этого основной текст обрабатывается за один проход. Для каждого числового слова выполняется переход автомата, а найденные образцы добавляются в массив ответов.

Для хранения переходов используется `std::unordered_map`, что позволяет эффективно работать с большим алфавитом без выделения памяти под все возможные значения. Для хранения вершин выбран `std::deque<Node>`, так как внутри автомата используются указатели на вершины, и их адреса должны оставаться стабильными.

Теоретическая сложность построения автомата составляет $O(M)$ в среднем, где M — суммарная длина образцов. Поиск выполняется за $O(N + Z)$ в среднем, где N — количество числовых слов текста, а Z — количество найденных вхождений. Память расходуется на автомат, очередь последних позиций и массив найденных ответов.